

AI와 페어 프로그래밍

풀스택 개발자의 생산성 비결

양효정

PyCon Busan 2026

오늘 이야기할 것

*페어 프로그래밍에서 배운 것들을,
AI와 함께 일하는 방식에 담았습니다.*

혼자 일하다, 페어를 만나다

혼자 일하던 시절

- 각자 맡은 영역이 달라 서로 코드 볼 기회 적음
- 피드백 주고받을 기회가 거의 없음
- AI 상용화 전 — 구글링이 유일한 동료

페어 프로그래밍

- XP(Kent Beck)의 핵심 실천법
- Driver + Navigator가 함께 코드 작성
- 핵심: **빠른 피드백**

피드백이 빨라질수록 학습이 빨라지고, 고치는 비용이 줄어든다

페어 프로그래밍을 만나다

둘이서 하면 시간이 두 배? — 아닙니다

시간이 두 배? — 아닙니다

- 실시간 논의로 작업 시간은 좀 늘지만
- 동료의 사고 흐름을 배울 수 있고
- 다른 개발 환경을 직접 보며 적용
- 배운 게 다음 작업에도 남는다

제 경험

- 어드민 페이지 로딩이 매우 오래 걸림
- 동료가 EXPLAIN 보는 법과 최적화 실험 방법 제시
- 이후 혼자서도 비슷한 최적화 가능

이후에는 혼자서도 비슷한 최적화를 할 수 있게 됐다

다시 혼자

페어의 환경을 재현할 수 있을까?

상황

이직 후 팀 구조상 각자 맡은 영역이 달라
페어 기회가 줄어들음



질문

AI가 페어 전체를 대체하진 않지만,
페어에서 얻었던 그 환경은 재현할 수 있지 않을까?



이번엔 내가 Navigator다

AI = Driver
코드 생성

/

나 = Navigator
방향 설정, 검증

* Claude Code 기준으로 설명하지만, 다른 AI 도구에도 적용 가능합니다

Layer 1: 컨텍스트

페어는 맥락 공유가 필수 → AI한테도 프로젝트 맥락을 공유하자

AI가 프로젝트를 모르면 실수가 시작된다

- "이 프로젝트는 Python 3.11이고, Airflow 2.x 쓰고..."
- 세션 열 때마다 같은 설명 반복
- 프로젝트가 10개면 10번 반복

Layer 1: 컨텍스트

AI 온보딩 문서

동료에게 프로젝트를 설명하듯, AI에게도 설명하자

예: Claude Code의 CLAUDE.md, Cursor의 .cursorrules 등

Airflow DAG 프로젝트

기술 스택

- Python 3.11, Apache Airflow 2.x (MWA)
- TaskFlow API 사용 (@task 데코레이터)

컨벤션

- DAG 파일: dags/ 디렉토리
- 테스트: pytest --no-header
- DAG ID 형식: {팀}_{도메인}_{작업}

자주 하는 실수

- XCom에 대량 데이터 저장하지 말 것 → S3 경유
- catchup=False 빠뜨리지 말 것

Layer 1: 컨텍스트

맥락이 쌓이면, 분리하자

예시: 실제 프로젝트에서 사용 중인 구조

```
memory/  
├─ feedback_*.md (9개)    ← AI에게 준 피드백/교정 모음  
├─ schema_*.md (5개)     ← DB 스키마  
├─ architecture.md        ← 프로젝트 구조  
├─ team.md                ← 팀 정보  
├─ operational-notes.md   ← 운영 경험/노하우  
└─ ... 총 40개+  
  
projects/  
├─ project-a/CLAUDE.md    ← 프로젝트별 온보딩 문서  
├─ project-b/CLAUDE.md  
└─ project-c/CLAUDE.md
```

하나의 파일에 다 넣지 말고, 맥락별로 분리하면 AI가 필요한 것만 골라서 읽는다

Layer 1: 컨텍스트

결과

BEFORE

매번 설명 반복
엉뚱한 코드 생성

AFTER

세션 열자마자 바로 작업
프로젝트에 맞는 코드 생성

피드백 타이밍: 설명하는 시간 → 0 — 즉시 피드백 가능한 상태

태스크 끝날 때마다 AI와 회고 → 맥락을 계속 쌓아가면 AI의 응답이 점점 정확해진다

Layer 2: 안전망

혼자 작업하면 '잠깐!' 해줄 사람이 없다 → 시스템이 대신 막아주자

위험한 상황은 시스템이 막는다

- AI가 생성한 명령어를 무심코 실행
- prod 환경에서 위험한 쿼리 실행할 뻔
- 리스크 고지 없이 혼자 진행해버림

CLAUDE CODE HOOK (자동 안전망)

AI가 명령 실행 전 자동으로 검사하는 Python 스크립트

```
# 위험한 패턴 정의 (정규식)
DANGEROUS_PATTERNS = [
    (re.compile(r"\bDROP\s+(DATABASE|TABLE)\b", re.I),
     "SQL DROP - 데이터가 영구 삭제됩니다"),
    (re.compile(r"\bgit\s+push\s+--force\b.*\b(main|master)\b", re.I),
     "Force push to main - 히스토리가 덮어씌워집니다"),
    # ... 20개+ 패턴
]

# AI가 실행하려는 명령을 검사
command = payload.get("tool_input", {}).get("command", "")
for pattern, reason in DANGEROUS_PATTERNS:
    if pattern.search(command):
        json.dump({"decision": "block", "reason": reason}, sys.stdout)
        return # 차단!
```

다른 도구는 pre-commit / shell wrapper로 비슷하게 구현 가능

Layer 2: 안전망 + 리뷰

코드 리뷰도 자동으로

동료가 리뷰해줬던 것 → AI 코드 리뷰어로 재현

- 내 프로젝트 컨벤션을 아는 리뷰어를 만든다
- 범용 AI 리뷰가 아니라, 프로젝트 맞춤 리뷰
- 팀 컨벤션이 있다면 그것부터 적용

피드백 타이밍: PR 후 리뷰 대기 → 코드 작성 직후로 앞당김

Layer 2: 안전망 + 리뷰

AI 코드도 클린하게

AI가 생성한 코드의 이슈 발생률: 사람 코드의 **1.7배**

로직 오류 1.75배 / 보안 취약점 1.57배

출처: CodeRabbit "AI Code Quality Report", 2025

- **테스트 커버리지** — 충분해야 AI가 작업 지침을 이해할 수 있다
- **일관된 컨벤션** — 코드와 문서가 불일치하면 AI가 오류를 빠르게 누적
- **클린한 코드베이스** — AI가 만든 코드도 사람이 짰 것처럼 일관적으로

코드베이스를 깨끗하게 유지하는 것이 AI 성능을 극대화하는 가장 확실한 방법

동료가 내가 놓친 걸 잡아줬다 → AI한테도 더블체크시키자

AI 작업 검증 (더블체크)

AI가 "끝났다"고 하면 → 진짜 끝났는지 자동 검증

자동 체크 항목

- 생성한 파일이 올바른 위치에 있는가?
- 불필요한 커밋이 섞이지 않았는가?
- 관련 파일 간 일관성이 유지되는가?

잡아낸 예시

- DAG schedule 변경했는데 catchup=False 빠뜨림
- 1년치 backfill이 쌓여 운영 DB 부하 발생 직전

배포 직전에 경고로 잡아낸다

Layer 3: 검증

결과

BEFORE

배포 후 장애로 발견

AFTER

배포 전에 사전 차단

피드백 타이밍: 배포 후 장애 → 배포 전 검증으로 앞당김

페어에서 가장 든든했던 '놓친 걸 잡아주는 동료' 역할을 SI가 해주는 셈

주의할 점

AI는 자신있게 거짓말합니다

```
# AI가 자신있게 추천한 코드
from airflow.providers.slack.operators import SlackWebhookOperator
# → 실제: from airflow.providers.slack.operators.slack_webhook import SlackWebhookOperator

from pandas_profiling import ProfileReport
# → deprecated, ydata-profiling으로 변경됨
```

"이해할 수 없는 코드는 절대 커밋하지 않는다"

Kent Beck의 XP: *Collective Code Ownership* — 코드와 책임은 팀이 공유

— AI는 팀원으로서 책임을 공유할 수 없다

그래서 컨텍스트, 안전망+리뷰, 검증이 필요합니다

더 나아가기

3가지 레이어가 단단하면, 그 위에서 확장할 수 있다

- 에이전트 = 독립된 전문 유닛 — 각각에게 선명한 맥락(Context)을 분리
- 무작정 머릿수를 늘리는 건 역효과 — 내가 감당할 수 있는 맥락의 총량을 넓혀가는 과정
- 1개로 시작 → 안정되면 하나씩 추가 → 지금 4개 동시 운용

내가 통제하고 이해할 수 있는 범위 안에서 점진적으로 확장하는 것

Layer 1~3도 같은 원리 — 한번에 다 만든 게 아니라, 필요할 때마다 하나씩 추가했습니다

오늘 당장 시작하기

1. **AI 온보딩 문서 만들기** — 프로젝트 루트에 기술 스택, 컨벤션, 주의사항 적기
이것만 해도 AI 응답 품질이 달라집니다
2. **위험 명령 차단 안전장치 하나 추가** — DROP, force push 등 치명적인 것만
20줄이면 됩니다
3. **AI 작업 결과 더블체크 습관** — 도구 없이 습관만으로도 시작 가능
"끝났다" 하면 한 번 더 확인

한번에 다 만들 필요 없습니다 — 하나씩 추가하면 됩니다

핵심 메시지

*빠른 피드백 루프,
그게 핵심이었습니다.*

*페어에서 배운 이 원리를 AI에 담으니,
혼자서도 잘 일할 수 있었습니다.*

Layer 1	맥락 공유가 필수	→	AI 온보딩 문서
Layer 2	안전망 + 실시간 리뷰	→	Hook + code-reviewer
Layer 3	놓친 걸 잡아주는 역할	→	더블체크

. 도구가 사람을 만든 게 아니라 , 사람이 도구를 제대로 쓴 거니까요 .

— Claude Code가 해준 말 중 인상 깊었던 한마디

결국 우리는 주어진 도구를 잘 씀으로써 더 잘 일할 수 있습니다.

세팅 공유

Claude Code 기준 예시 — 자기 환경에 맞게 응용해보세요

GitHub



github.com/yangchoi/pycon-2026

LinkedIn



양효정

AI 온보딩 문서 Hook code-reviewer 더블체크

감사합니다!

Q&A